

Lista de Questões Práticas — AEDS II

Árvores, Tabelas Hash, TRIE e Estruturas Híbridas

Questões 1 a 39 — Estilo de Prova (somente enunciados)

Todas as questões abaixo seguem o estilo de prova: o professor fornece estruturas prontas (tipos de nó, árvore, hash, TRIE etc.) e pede para implementar funções/métodos específicos (geralmente `boolean` ou `int`) e analisar a complexidade. **Não coloque as respostas aqui.**

Questão 1 • Altura limitada por log da quantidade de nós

Considere as classes:

```
class No {  
    int elemento;  
    No esq, dir;  
}  
  
class Arvore {  
    private No raiz;  
}
```

Implemente o método `boolean isMax(double valor)` em `Arvore`, que verifica se a árvore tem altura menor ou igual a $\text{valor} * \log_2(\text{qtdNos})$, onde `qtdNos` é o número total de nós da árvore. Não existem métodos prontos para contar nós ou altura. Analise a complexidade em função de `qtdNos`.

Questão 2 • Verificar se a árvore é ABB e AVL ao mesmo tempo

Ainda com `No` e `Arvore`, implemente `boolean isAVL()` que verifica se a árvore é simultaneamente:

- uma ABB (todos os valores da subárvore esquerda de um nó são menores que o elemento do nó, e todos os da direita são maiores);
- uma AVL (para todo nó, a diferença entre as alturas das subárvores esquerda e direita, em módulo, é no máximo 1).

Você pode criar funções auxiliares privadas. Analise a complexidade.

Questão 3 • Verificar se a árvore é ABB

Com as mesmas classes, implemente o método `boolean ehABB()` que retorna `true` se a árvore respeita a propriedade de ABB para todos os nós, e `false` caso contrário. Discuta uma forma de implementar sem copiar os elementos para um vetor. Analise a complexidade.

Questão 4 • Contar nós com valor par

Implemente em `Arvore` o método `int contarPares()` que retorna a quantidade de nós cujo `elemento` é um número par. O método deve percorrer toda a árvore. Analise a complexidade em função do número de nós.

Questão 5 • Verificar se a árvore é estritamente binária

Uma árvore binária é estritamente binária se todo nó possui 0 ou 2 filhos. Implemente `boolean ehEstritamenteBinaria()` que retorna `true` se a árvore for estritamente binária e `false` caso contrário. Analise a complexidade.

Questão 6 • Percurso em ordem gerando string

Implemente o método `String emOrdem()` que retorna uma string com os elementos da árvore em ordem crescente (percurso em ordem: esquerda, raiz, direita), separados por espaço. Não use estruturas auxiliares prontas, apenas recursão e concatenação. Analise a complexidade.

Questão 7 • Soma de chaves em intervalo [a, b] em uma ABB

Suponha que `Arvore` é uma ABB válida. Implemente `int somaIntervalo(int a, int b)` que devolve a soma de todos os elementos `x` tais que `a <= x <= b`. Use a propriedade de ABB para evitar percorrer ramos desnecessários. Analise a complexidade no pior caso e em um caso médio com árvore balanceada.

Questão 8 • Verificar se duas árvores são espelho uma da outra

Considere duas instâncias de `Arvore`. Implemente `boolean ehEspelho(Arvore outra)` que retorna `true` se a árvore atual e a árvore `outra` forem espelho uma da outra (para cada nó na primeira há um nó correspondente na segunda em posição invertida esquerda/direita com o mesmo valor). Analise a complexidade.

Questão 9 • Verificar se uma árvore é AVL e tem altura limitada

Considere:

```
class NoAVL {
    int elemento;
    NoAVL esq, dir;
    int altura;
}

class ArvoreAVL {
    private NoAVL raiz;
}
```

Implemente `boolean estaAVLComAlturaMax(int h)` que retorna `true` se a árvore é uma AVL válida e tem altura menor ou igual a `h`. Pode assumir que `altura` em cada nó já está correta. Analise a complexidade.

Questão 10 • Recalcular alturas em árvore AVL

Reutilizando `NoAVL` e `ArvoreAVL`, implemente `void recalcularAlturas()` que percorre toda a árvore e atualiza o campo `altura` de cada nó (altura de folha = 0, árvore vazia = -1). Analise a complexidade.

Questão 11 • Contar folhas em uma árvore 2–3–4

Considere:

```
class No234 {
    int qtdChaves;           // 1, 2 ou 3
    int chave1, chave2, chave3;
    No234 f0, f1, f2, f3;    // qtdChaves+1 filhos válidos
}

class Arvore234 {
    No234 raiz;
}
```

Implemente `int contarFolhas()` que retorna o número de nós folha da árvore. Analise a complexidade.

Questão 12 • Verificar se todas as folhas de uma 2–3–4 estão na mesma altura

Usando `No234` e `Arvore234`, implemente `boolean folhasMesmaAltura()` que retorna `true` se todas as folhas da árvore estão na mesma altura (propriedade essencial de árvores 2–3–4) e `false` caso contrário. Analise a complexidade.

Questão 13 • Verificar propriedades básicas de árvore alvinegra

Considere:

```
class NoAN {
    boolean cor;           // true = preto, false = vermelho
    int elemento;
    NoAN esq, dir;
}

class ArvoreAlvinegra {
    private NoAN raiz;
}
```

Implemente `boolean verificaCores()` que retorna `true` se:

- (a) nenhum nó vermelho tem filho vermelho;
- (b) o número de nós pretos em qualquer caminho da raiz até uma folha nula é o mesmo.

Caso alguma condição seja violada, retorne `false`. Analise a complexidade.

Questão 14 • Mesma quantidade de nós brancos nas subárvores da raiz

Agora considere que `cor = true` representa nó branco e `cor = false` representa nó preto. Implemente em `ArvoreAlvinegra` o método `boolean raizEquilibradaBranca()` que verifica se as subárvores esquerda e direita da raiz possuem a mesma quantidade de nós brancos. Se

a árvore estiver vazia, retorne `true`. Analise a complexidade.

Questão 15 • Todos os nós com mesma quantidade de nós brancos dos dois lados

Ainda com `NoAN` e `ArvoreAlvinegra`, implemente `boolean equilibradaBrancaGlobal()` que retorna `true` se, para todo nó `x`, a quantidade de nós brancos na subárvore esquerda de `x` for igual à quantidade de nós brancos na subárvore direita de `x`. Caso exista algum nó que não satisfaça essa igualdade, retorne `false`. Analise a complexidade.

Questão 16 • Mesmo número de nós brancos em todos os caminhos

Implemente em `ArvoreAlvinegra` o método `boolean mesmosBrancoNosCaminhos()` que verifica se todos os caminhos da raiz até qualquer folha possuem a mesma quantidade de nós brancos. Árvore vazia deve ser considerada verdadeira. Analise a complexidade.

Questão 17 • Busca de palavra exata em TRIE

Considere a TRIE para letras 'a'...'z':

```
class NoTrie {
    NoTrie[] filho = new NoTrie[26];
    boolean fim;
}

class ArvoreTrie {
    private NoTrie raiz;
}
```

Implemente `boolean existePalavra(String s)` que retorna `true` se a palavra `s` estiver armazenada na TRIE (caminho existe e nó final tem `fim = true`) e `false` caso contrário. Analise a complexidade em função do tamanho de `s`.

Questão 18 • Inserção de palavra em TRIE

Implemente `void inserir(String s)` em `ArvoreTrie`, que insere a palavra `s` na estrutura, criando nós quando necessário e marcando `fim = true` no nó correspondente ao último caractere. Analise a complexidade em função de `|s|`.

Questão 19 • Contar total de palavras armazenadas em TRIE

Implemente `int contarPalavras()` que retorna o número total de palavras armazenadas (quantidade de nós com `fim = true`) em toda a TRIE. Analise a complexidade em função do número de nós da TRIE.

Questão 20 • Contar palavras que começam com um prefixo

Implemente `int contarPrefixo(String pref)` que devolve quantas palavras armazenadas na TRIE começam com o prefixo `pref`. O método deve chegar ao nó que representa o final

do prefixo e, a partir dele, percorrer a subárvore contando nós com `fim = true`. Analise a complexidade.

Questão 21 • Existe palavra com um dado prefixo

Implemente `boolean existeComPrefixo(String pref)` que retorna `true` se existir pelo menos uma palavra armazenada que começa com `pref`, e `false` caso contrário. Analise a complexidade em função de `|pref|` e do número de palavras que compartilham esse prefixo.

Questão 22 • Existe palavra que começa com prefixo e termina com sufixo

Implemente `boolean existeComPrefixoESufixo(String prefixo, String sufixo)` que retorna `true` se existir alguma palavra na TRIE que começa com `prefixo` e termina com `sufixo`. Explique como localizar o nó do prefixo e, a partir dele, verificar o sufixo nas palavras abaixo. Analise a complexidade.

Questão 23 • Existe palavra com prefixo, mas sem determinado sufixo

Implemente `boolean existeComPrefixoESemSufixo(String prefixo, String sufixoProibido)` que retorna `true` se existir ao menos uma palavra que começa com `prefixo` e não termina com `sufixoProibido`. Se todas as palavras que começam com o prefixo terminarem exatamente com o sufixo proibido, retorne `false`. Analise a complexidade.

Questão 24 • Verificar se uma TRIE compactada é PATRICIA válida

Considere:

```
class NoPatricia {
    String rotulo;          // trecho de caracteres neste arco
    NoPatricia[] filhos;    // filhos (vetor ou lista)
    boolean fim;
}

class ArvorePatricia {
    private NoPatricia raiz;
}
```

Implemente `boolean ehPatricia()` que verifica se não existe nó interno com rótulo vazio e nem nó interno com apenas um filho (ou seja, nenhuma compressão adicional seria possível). Analise a complexidade.

Questão 25 • Estrutura doidona 1: hash + lista + ABB de inteiros

Considere:

```
class NoABB {
    int elemento;
    NoABB esq, dir;
}
```

```
class Celula {
    int indice;
    NoABB raiz;
    Celula prox;
}

class T1 {
    int[] tabela; // área principal
}

class T2 {
    Celula primeiro; // área de reserva encadeada
}

class EstruturaDoidona {
    private T1 t1;
    private T2 t2;
}
```

Suponha `hash(int x)` e `rehash(int x)` já implementadas. Implemente `void inserir(int x)`, `boolean pesquisar(int x)`, `boolean remover(int x)` e `void mostrar()` em `EstruturaDoidona`. Colisões em `T1` devem ir para uma ABB dentro de uma célula em `T2`. Analise a complexidade.

Questão 26 • Estrutura doidona 2: árvore de caracteres + hash + lista

Considere:

```
class CelulaLista {
    String palavra;
    CelulaLista prox;
}

class TabelaHash {
    CelulaLista[] tabela;
}

class NoChar {
    char c;
    TabelaHash t1;
    NoChar esq, dir;
}

class DicionarioDoidona2 {
    private NoChar raiz;
}
```

Cada `NoChar` representa a primeira letra da palavra, e sua `TabelaHash` armazena palavras que começam com essa letra. Suponha `hashPalavra(String s)` pronta. Implemente `boolean pesquisar(String palavra)` que: (1) usa a árvore de caracteres para localizar o nó da primeira letra; (2) pesquisa a palavra na hash `t1`. Analise a complexidade.

Questão 27 • Hash com área de reserva AVL — versão com Lista e Árvore

Considere:

- T1: tabela hash de tamanho m (vetor de inteiros);
- T2: lista encadeada, cada célula com raiz de uma árvore AVL contendo os inteiros que colidiram.

Suponha `hash(int x)`, `rehash(int x)` e uma classe `ArvoreAVL` básica. Implemente em `EstruturaAVLHash`: `inserir(int x)`, `boolean pesquisar(int x)`, `boolean remover(int x)` e `void mostrar()`, usando T1 e T2. Analise a complexidade.

Questão 28 • Hash com área de reserva AVL — versão com NoAVL e Celula

Agora, sem classe `ArvoreAVL` externa:

```
class NoAVL {
    int elemento;
    NoAVL esq, dir;
    int altura;
}

class Celula {
    int indiceReserva;
    NoAVL raizAVL;
    Celula prox;
}

class EstruturaAVLHash2 {
    private int[] T1;
    private Celula inicioT2;
}
```

Suponha `hash(int x)` e `rehash(int x)` prontas. Implemente `inserir(int x)`, `boolean pesquisar(int x)`, `boolean remover(int x)` e `void mostrar()` manipulando diretamente `NoAVL` e `Celula`. Analise a complexidade.

Questão 29 • Hash com área de reserva em árvore 2–3–4

Considere:

```
class Arvore234 {
    No234 raiz;
    // métodos básicos
}

class Hash234 {
    int[] T1;
    Arvore234 reserva;
}
```

T1 é tabela hash de inteiros e `reserva` é árvore 2–3–4 com colisões. Suponha `hash(int x)` implementada. Implemente `inserir(int x)`, `boolean pesquisar(int x)`, `boolean remover(int x)` e `void mostrar()` em `Hash234`. Analise a complexidade quando há poucas e quando há muitas colisões.

Questão 30 • Hash com área de reserva em árvore alvinegra

Considere:

```
class ArvoreAN {
    private NoAN raiz;
    // inserção/busca/remoção já balanceadas
}

class HashAlvinegra {
    int[] T1;
    ArvoreAN reserva;
}
```

Sendo T1 uma hash de inteiros, colisões vão para **reserva**. Implemente `inserir(int x)`, `boolean pesquisar(int x)`, `boolean remover(int x)` e `void mostrarEmOrdem()` em `HashAlvinegra`. Analise a complexidade de busca e inserção.

Questão 31 • Tabela hash de palavras com TRIE como área de reserva

Considere:

```
class NoTrie {
    NoTrie[] filho = new NoTrie[26];
    boolean fim;
}

class ArvoreTrie {
    private NoTrie raiz;
}

class HashTrie {
    String[] T1;
    ArvoreTrie reserva;
}
```

T1 tenta armazenar diretamente palavras; colisões vão para a TRIE **reserva**. Suponha `hashPalavra(String s)` pronta. Implemente `inserir(String palavra)`, `boolean pesquisar(String palavra)`, `boolean remover(String palavra)` (remoção lógica) e `void mostrar()`. Analise a complexidade.

Questão 32 • Hash de prefixos com TRIE PATRICIA na área de reserva

Considere:

```
class ArvorePatricia {
    private NoPatricia raiz;
}

class HashPatricia {
    String[] T1;
    ArvorePatricia reserva;
}
```

T1 armazena palavras; colisões vão para a PATRICIA **reserva**. Suponha `hashPalavra(String s)` pronta. Implemente `inserir(String palavra)`, `boolean pesquisar(String palavra)` e `boolean remover(String palavra)` em `HashPatricia`.

Analise a complexidade, comentando o efeito da compressão da PATRICIA.

Questão 33 • Árvore de categorias com hash de produtos e AVL de reserva

Considere:

```
class NoProduto {
    int codigo;
    String nome;
    NoProduto esq, dir;
    int altura;
}

class HashProdutos {
    int[] T1;
    NoProduto reserva;
}

class NoCategoria {
    String nome;
    HashProdutos produtos;
    NoCategoria esq, dir;
}

class ArvoreCategorias {
    private NoCategoria raiz;
}
```

Cada categoria é um nó de ABB de categorias (ordenadas por nome) e possui HashProdutos com AVL de reserva. Suponha hashCode(int c) e rotações AVL prontas. Implemente: `inserirProduto(String categoria, int codigo, String nome)`, `boolean pesquisarProduto(String categoria, int codigo)`, `boolean removerProduto(String categoria, int codigo)` e `void mostrarProdutos()`. Analise a complexidade da busca.

Questão 34 • Árvore de árvore: categorias e ABB de produtos

Versão sem hash:

```
class NoProduto {
    int codigo;
    String nome;
    NoProduto esq, dir;
}

class NoCategoria {
    String nome;
    NoCategoria esq, dir;
    NoProduto raizProdutos;
}

class ArvoreCategoriasSimples {
    private NoCategoria raiz;
}
```

Implemente `NoProduto buscarProduto(String categoria, int codigo)` que procura primeiro a categoria na ABB de categorias e depois o produto na ABB de produtos. Analise

a complexidade em função das alturas das duas árvores.

Questão 35 • Árvore doidona com regra par/ímpar para inserção

Considere:

```
class No {
    int elemento;
    No esq, dir;
}

class ArvoreDoidona {
    private No raiz;
}
```

Regra de inserção:

- se x é par: regra de ABB normal (x menor vai à esquerda);
- se x é ímpar: regra invertida (x menor vai à direita).

Implemente `void inserir(int x)` obedecendo a regra e discuta as dificuldades de implementar uma busca correta nessa árvore. Analise a complexidade da inserção.

Questão 36 • Percurso por níveis em árvore AVL usando fila

Com `NoAVL` e `ArvoreAVL`, implemente `void imprimirPorNiveis()` que imprime os elementos da árvore em ordem de níveis (breadth-first). Utilize uma fila implementada por você (por exemplo, encadeada). Analise a complexidade em função do número de nós.

Questão 37 • TRIE de palavras com hash de documentos (índice invertido)

Considere:

```
class NoDoc {
    String id;
    NoDoc prox;
}

class HashDocs {
    NoDoc[] tabela;
}

class NoTrieDocs {
    NoTrieDocs[] filho = new NoTrieDocs[26];
    boolean fim;
    HashDocs docs;
}

class ArvoreTrieDocs {
    private NoTrieDocs raiz;
}
```

Implemente `void listarDocumentosPrefixo(String prefixo)` que, dado um prefixo, lista todos os ids de documentos que contêm alguma palavra iniciada por esse prefixo. Analise a complexidade.

Questão 38 • Estrutura híbrida: chaves pares em AVL, ímpares em TRIE

Considere:

```
class EstruturaParImpar {
    private ArvoreAVL arvorePares;
    private ArvoreTrie trieImpares;
}
```

Chaves pares vão para a `ArvoreAVL`, ímpares para a `ArvoreTrie` (usando representação decimal como string). Implemente `void inserir(int x)` e `boolean pesquisar(int x)`, decidindo a estrutura com base na paridade de `x`. Analise a complexidade em função da altura da AVL e do número de dígitos de `x`.

Questão 39 • Índice de busca com TRIE e listas de documentos

Considere:

```
class NoDocSimples {
    String id;
    NoDocSimples prox;
}

class NoTrieIndice {
    NoTrieIndice[] filho = new NoTrieIndice[26];
    boolean fim;
    NoDocSimples listaDocs;
}

class ArvoreIndice {
    private NoTrieIndice raiz;
}
```

Implemente `boolean existeDocComPalavraPrefixo(String prefixo, String idDoc)` que retorna `true` se existir alguma palavra armazenada que começa com `prefixo` e que esteja associada ao documento `idDoc` na lista de documentos. Caso contrário, retorna `false`. Analise a complexidade.